Tiago Autonomous Exploration and Manipulation

Autonomous and Mobile Robotics - Project

Francesco Cenni Giulio Mazzaroppi Giacomo Pezzolati

francesco.cenni3@studio.unibo.it
giulio.mazzaroppi@studio.unibo.it
giacomo.pezzolati@studio.unibo.it

University of Bologna

Academic Year 2025/2026

February 8, 2026

Overview

1. Introduction
2. Task 1: Exploration
3. Task 2: Navigation
4. Task 3: Manipulation

Project Overview

- **Objective:** Develop a complete autonomous system for the TiaGo robot combining navigation and manipulation skills.
- **Tasks:**
  1. Map Generation (SLAM).
  2. Autonomous Localization & Exploration.
  3. Pick and Place of specific ArUco cubes.

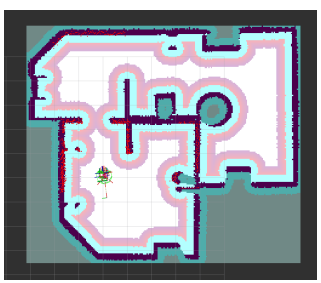


Figure 1: Map Generation in RViz.

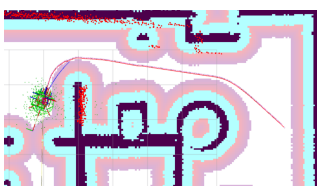


Figure 2: Autonomous Localization & Exploration

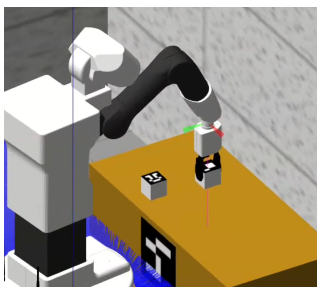


Figure 3: Pick and Place of specific ArUco cubes

System Architecture & Design

Development Stack

- **Framework:** ROS2 Humble Hawksbill.
- **Simulation:** Gazebo 11 (Physics & Environment).
- **Visualization:** RViz2 (Real-time sensor data & markers).

Architectural Pattern

We adopted a modular **Policy vs. Actuator** approach:

- **Main Policy (Coordinators):** High-level nodes managing the task logic and state machine.
- **Generalized Actuators:** Reusable skills (e.g., `map_navigator`, `head_motion`) agnostic of the specific task.
- **Handshake Protocol:** Coordination is handled via strictly defined Topics, Services, and Actions to ensure robust synchronization[a].

[a] Custom messages and service definitions are isolated in a dedicated **ament_cmake** package: `tiago_exam_interfaces`.

Package Organization: `tiago_exam`

The Project is entirely accomplished inside the `tiago_exam` package. This is a **ament_python** package structured as follows:

- **launch:** Contains three launch files to start each respective task environments.
- **tiago_exam:** Organized into sub-packages for each task and a shared utilities package.
- **maps:** Stores the generated map from Task 1. It contains the `group29_map.yaml` and `group29_map.pgm` files.
- **data:** Contains the csv file for the aruco markers records used in task2 and task3.
- **config:** Stores RViz configuration files and map data for easy access across tasks.
- **temp:** Contains the debug map images for the exploration surface and reference frames.

Task 1: Autonomous Map Generation

Objective

Perform a **full autonomous exploration** of **an unknown environment** to generate a consistent 2D occupancy grid map using SLAM.

Environmental Challenges

The testing environment presents specific difficulties that standard configuration could not handle:

- **Complex Geometry:** Narrow passages and obstacles requiring precise maneuvering.
- **Lighting Artifacts:** Strong shadows created false positive obstacles in the LIDAR readings (ghost objects).

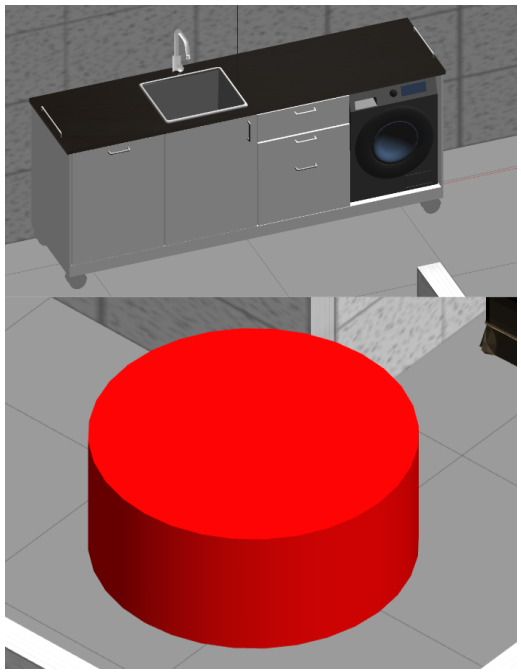


Figure 4: Environment in Gazebo 11.

Task 1: Exploration Strategy

To achieve efficient coverage, we integrated a C++ node based on `m-explore`, enhancing the standard frontier exploration logic.

Optimized Frontier Selection Logic

Instead of a greedy approach, we tuned the cost function parameters to prioritize frontiers that are:

1. **Closer** to the robot's current pose (minimizing travel time).
2. Located in **larger unexplored voids** (maximizing information gain).

Completion Workflow:
Upon exhaustion of the whitelist frontiers, the robot automatically navigates back to the **Home Position** and triggers the map saver, storing the result in the `maps/` directory.

Task 1: Safety Configuration

Safety Measure: Pre-Exploration Configuration

Before moving, the system executes a mandatory **Tuck Arm** procedure. Folding the arm into a safe configuration minimizes the robot's footprint, preventing collisions with low obstacles or door frames during autonomous navigation.

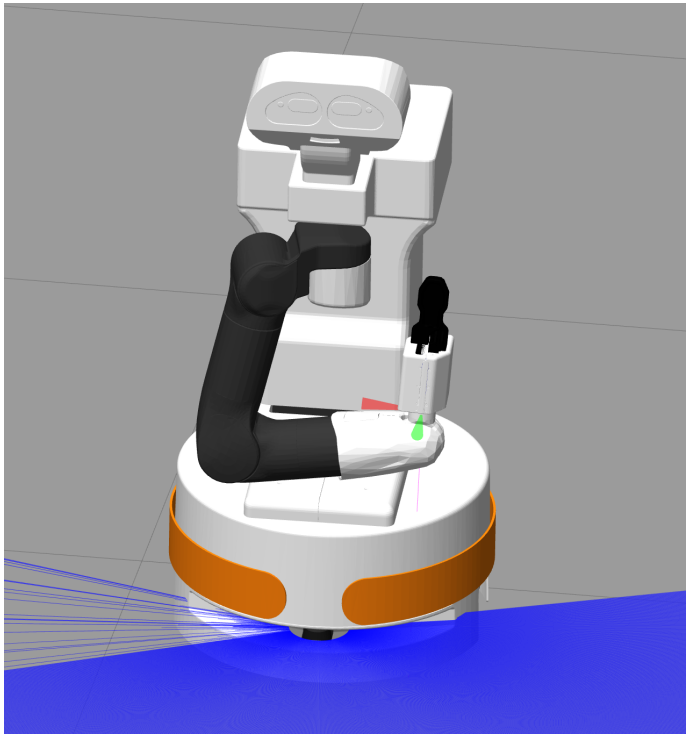


Figure 5: Tuck Arm Configuration for Safe Navigation

This configuration is also considered in the navigation for Task 2 and Task 3 as safe Pose.

Task 1: System Configuration

Sensor & Navigation Tuning
We refined the navigation stack parameters to address the environmental noise:

- **LIDAR Raytracing:** Adjusted strictly to filter out shadow reflections and prevent "ghost obstacles" on the map.
- **Motion Profile:** Smoothed acceleration limits to ensure stable mapping and prevent odometry drift.
- **Cost Map Parameters:**

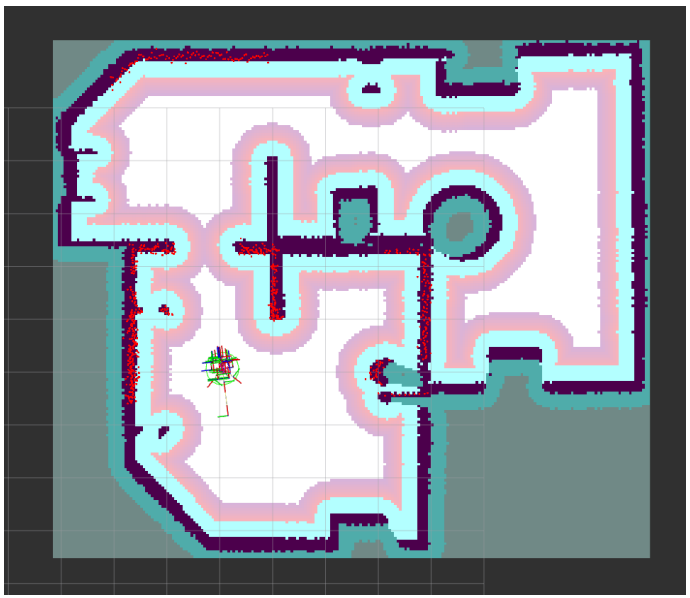


Figure 6: Generated Occupancy Grid

Task 1: Exploration Video Result

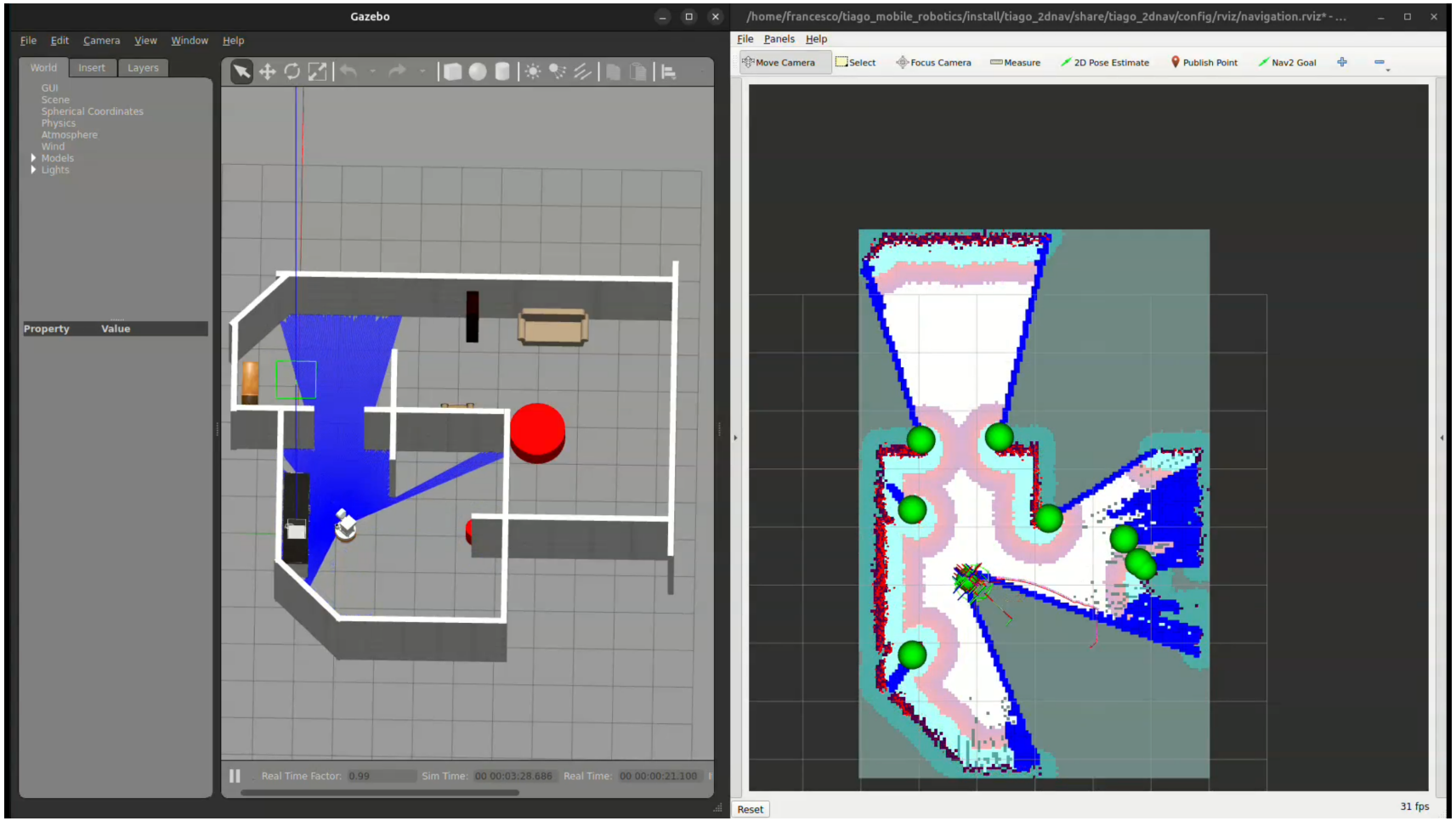


Figure 7: Exploration Task Execution (Click to play in browser)

Task 2: Autonomous Map Navigation

Objective

Perform a **full autonomous navigation** of a **known environment** using the previously generated 2D occupancy grid map to find and record the pose of specific Aruco markers.

- **Pick Station Marker:** Aruco Marker ID 26, size 0.25m.
- **Place Station Marker:** Aruco Marker ID 238, size 0.25m.

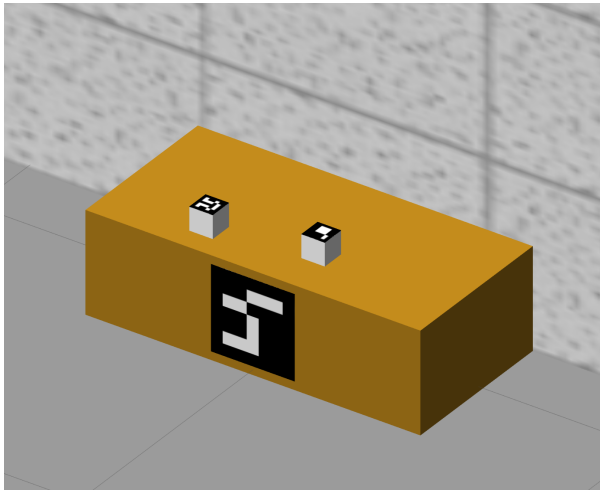


Figure 8: Pick Station Marker 26.

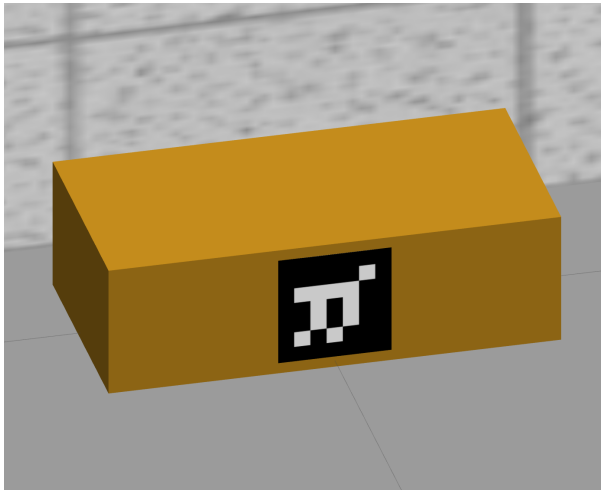


Figure 9: Place Station Marker 238.

Task 2: Navigation Trade-offs

Operational Challenges

ArUco markers are attached to tables that may be located in corners or occluded areas.

Strategic Trade-off: Search Logic

Exhaustive Perimeter Search

To locate the tables, we evaluated an *exhaustive perimeter search* (wall-following) to systematically scan every corner.

Probabilistic Heuristic (Naive Explorer)

However, we opted for a **probabilistic heuristic**. By generating safe random goals within the free space, the robot covers the environment more rapidly than tracing the full perimeter, maintaining high detection reliability with lower execution time.

Task 2: System Architecture & Orchestration

The complex behavior required for Task 2 is managed by a central **Coordinator Node** that acts as the high-level decision maker.

The Coordinator Design

- **Finite State Machine (FSM):** Strict transition logic between *Exploring*, *Tracking*, and *Inspecting* states.
- **Concurrency Handling:** Utilizes `ReentrantCallbackGroup` and `MultiThreadedExecutor`. This allows the node to process sensory interrupts (e.g., seeing a marker) immediately, even while blocking navigation actions are active.
- **Interrupt Logic:** The standard exploration loop is designed to be preempted instantly upon ArUco detection.

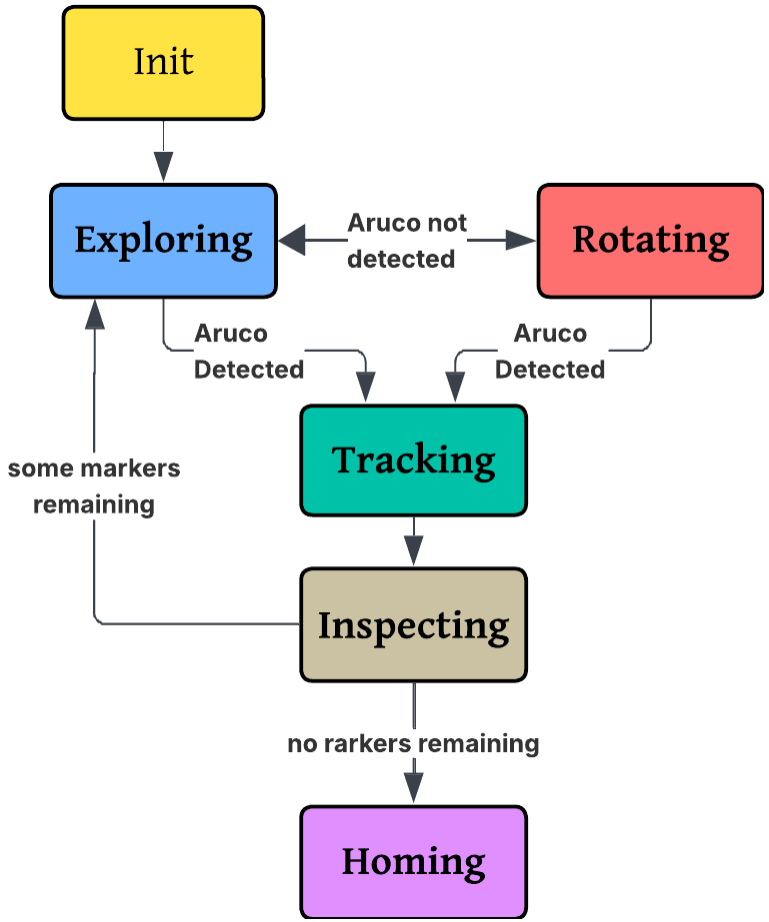


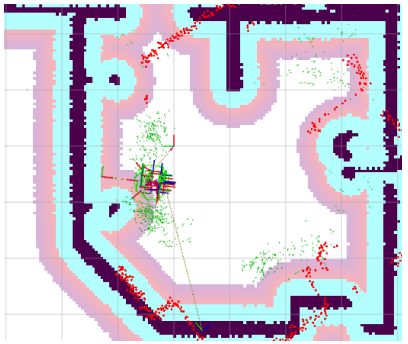
Figure 10: State Machine Logic

Task 2: Global Localization

Since the robot starts in an unknown location on the map, we implemented a global localization routine exploiting the Adaptive Monte Carlo Localization (AMCL).

The Converge Strategy:

1. **Particle Spread:** Trigger `/reinitialize_global_localization` to distribute AMCL particles uniformly across the map.
2. **Data Gathering:** The robot executes a 360° in-place rotation at 1.0 rad/s to feed the laser scan matcher.
3. **Covariance Check:** The routine continuously monitors the pose covariance matrix, ensuring it falls below strict thresholds before allowing any movement.



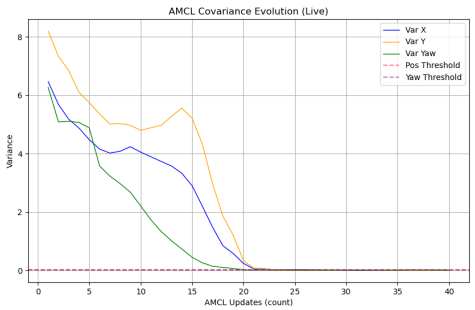


Figure 11: Covariance Reduction Over Time

Task 2: Exploration Strategy

After the localization, the Coordinator Node activate the following nodes:

Generalized Nodes

- **Aruco Marker Detector:** The node continuously monitors the camera feed for the station markers.
- **Explorator:** The node processes the occupancy grid map to create a "whitelist" of safe navigation targets, ensuring the robot only attempts to navigate to locations that are free of obstacles.
- **Navigator:** The robot continuously samples from the "whitelist" of safe exploration goals. After reaching each goal, the robot performs a 360° rotation to improve its environmental awareness.

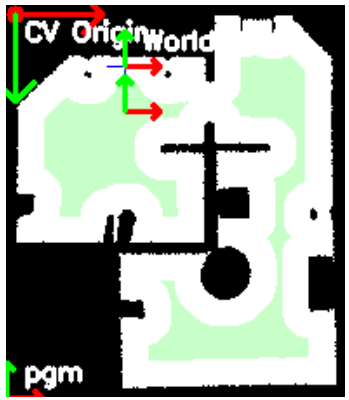


Figure 12: Clearance-Based Goal Sampling

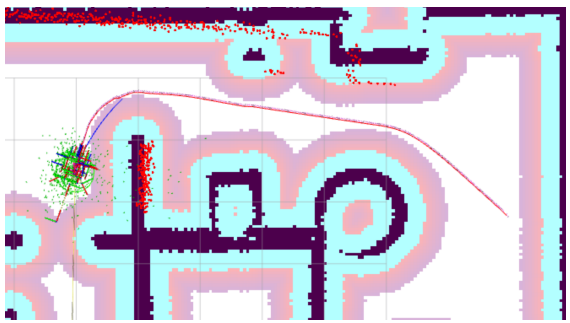


Figure 13: Random Safe Goal Navigation

Task 2: Exploration Strategy - Contd.

Exploration Strategy

- **Tracking:** Once an ArUco marker is detected, the robot is brought to a specific distance from the marker, while continuously facing it by adjusting the camera tilt and pan angles.
- **Inspecting:** Once the robot is correctly positioned, it transitions to the Inspecting state, where the marker and robot Pose is saved in a csv file for later use.
- **Homing:** After all the markers are detected, the robot automatically navigates back to the Home Position.

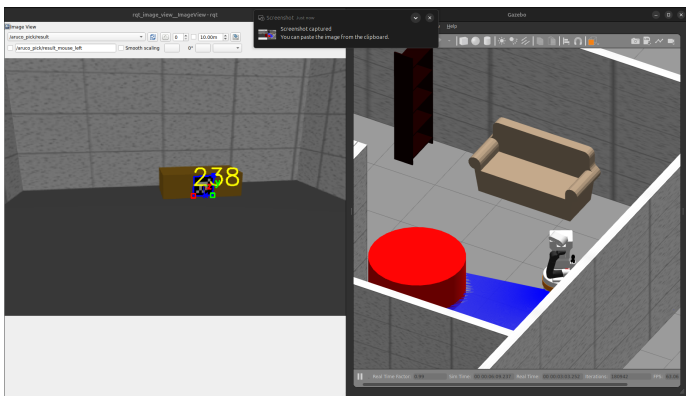


Figure 14: Aruco Tracking Behavior

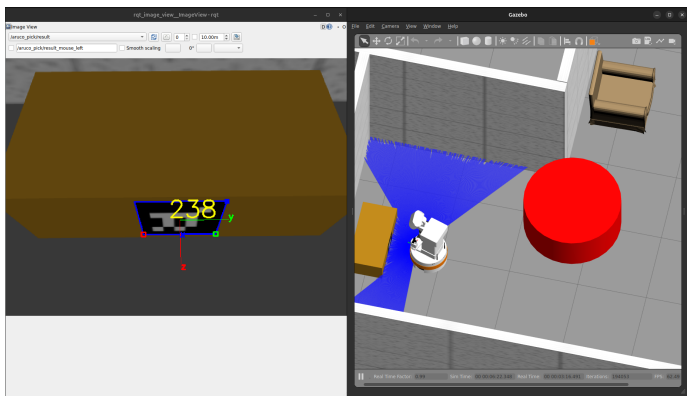


Figure 15: Aruco Inspecting Behavior

Task 2: Safety Navigation

Map Processing Pipeline

Using OpenCV on the occupancy grid map:

1. **Safety Mapping:** Applied `cv2.distanceTransform` to calculate the distance of every pixel to the nearest obstacle.
2. **Filtering:** Created a "Whitelist Mask" of pixels where:

$$d_{obstacle} > (R_{robot} + \text{Clearance}_{0.7m})$$

3. **Goal Selection:** Randomly samples coordinates from the whitelist and converts them from Image Frame (pixels) to World Frame (meters).

Result: The robot never attempts to navigate into walls or narrow traps.

Task 2: Navigation Video Result

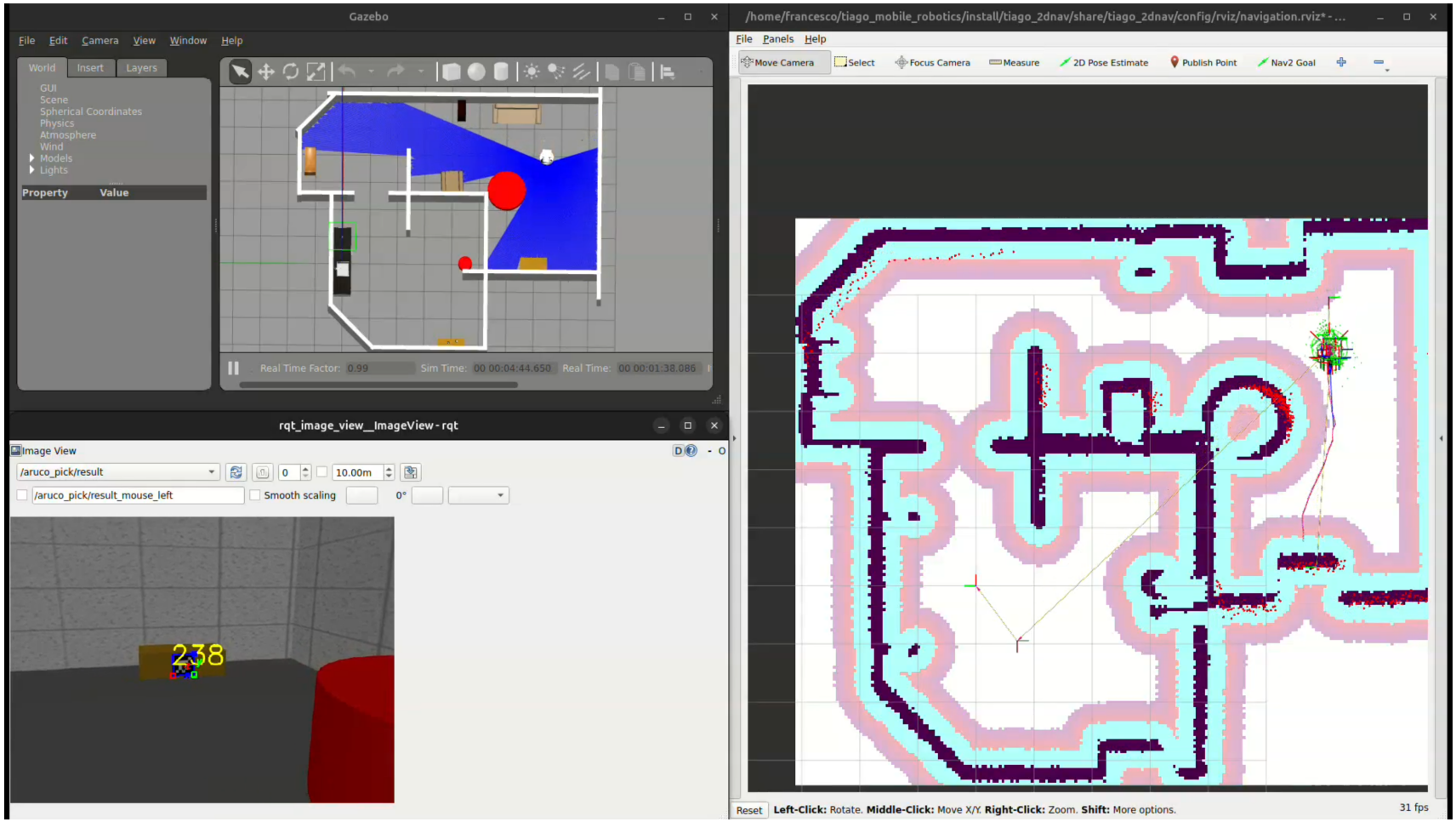


Figure 16: Navigation Task Execution (Click to play in browser)

Task 3: Pick & Place

Objective

Execute a precise **Pick & Place** operation leveraging on:

- The map generated in Task 1.
- The marker locations identified in Task 2.

Operational Constraints

The manipulation task requires handling strict mechanical and spatial tolerances:

- **Grasp Tolerance:** The parallel gripper has minimal mechanical compliance. Even slight ArUco pose estimation errors (*homography noise*) can cause the fingers to collide with the object rather than grasping it.
- **Kinematic Reachability:** Placement success is strictly coupled with **Base Positioning**. If the robot stops too far from the table, the arm's maximum reach is insufficient, causing the cube to be released into the void.
- **Deployment Safety:** Transitioning the arm from the *Safe Travel Configuration* to the *Work Configuration* near obstacles poses a high risk of collision with the table edge.

Task 3: Coordinator

The `coordinator_task3` node orchestrates the sequence, bridging the data gathered in Task 2 with the actuation logic.

Execution Workflow

1. **Data Ingestion:** Parses `marker_locations.csv` to retrieve the exact pose of the tables targets found during Task 2 exploration.
2. **State Sequence:** Enforces the required order of operations:

$$\text{Localize} \xrightarrow{\text{Navigate to Pick}} \text{Pick Cube ID 63} \xrightarrow{\text{Navigate to Place}} \text{Place Cube ID 63}$$

$$\text{Next Target} \xrightarrow{\text{Navigate to Pick}} \text{Pick Cube ID 582} \xrightarrow{\text{Navigate to Place}} \text{Place Cube ID 582}$$

3. **Visual Feedback:** Publishes dynamic TF frames (`target_cube_frame`) to visualize the manipulation target in RViz in real-time.

Once all targets cubes are processed, the coordinator commands the robot to return to the **Home Position**, signaling Task 3 completion.

Task 3: Manipulator

The `manipulator` node orchestrates the sequence, handling the actual arm movements and gripper actions on the basis of the `coordinator_task3` commands. The commands are in the form of:

- **Pick Command:** `'PICK_<cube_id>'`.
- **Place Command:** `'PLACE_<cube_id>'`.

This commands issue the manipulator node to execute the corresponding manipulation sequence.

PICK_<63>

$$\text{Safe Pose} \xrightarrow{\text{Idle for aruco pose update}} \text{Pre_Grasp} \rightarrow \text{Grasp} \rightarrow \text{Post_Grasp} \rightarrow \text{Safe Pose}$$

PLACE_<63>

$$\text{Safe Pose} \xrightarrow{\text{Idle for place station}} \text{Pre_Release} \rightarrow \text{Release} \rightarrow \text{Post_Release} \rightarrow \text{Safe Pose}$$

Task 3: Coordinator & Manipulator State Machines

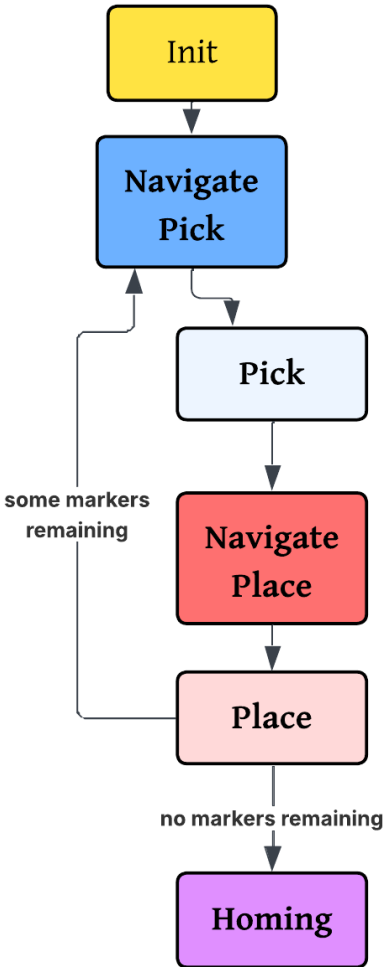


Figure 17: Coordinator

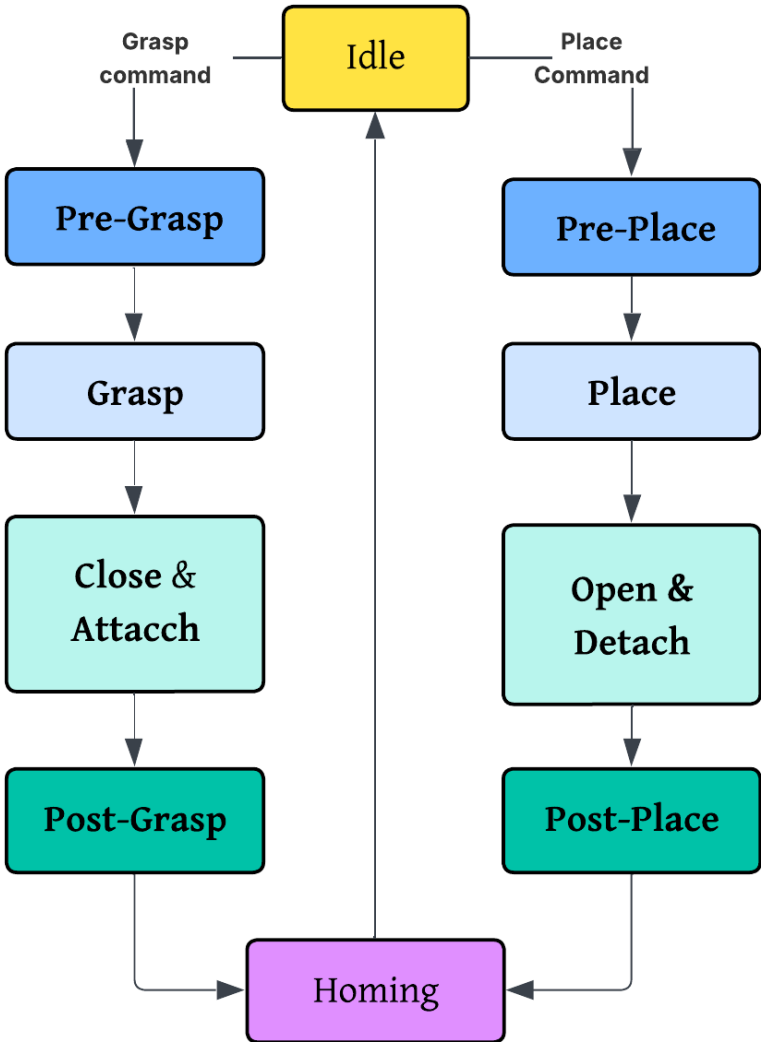


Figure 18: Manipulator

Task 3: Marker Perception

Perception Sequence

Head Scanning & Latching

- While the aruco detector is running the robot head **tilts down** when the station is reached.
- The system caches the **latest valid transform**.

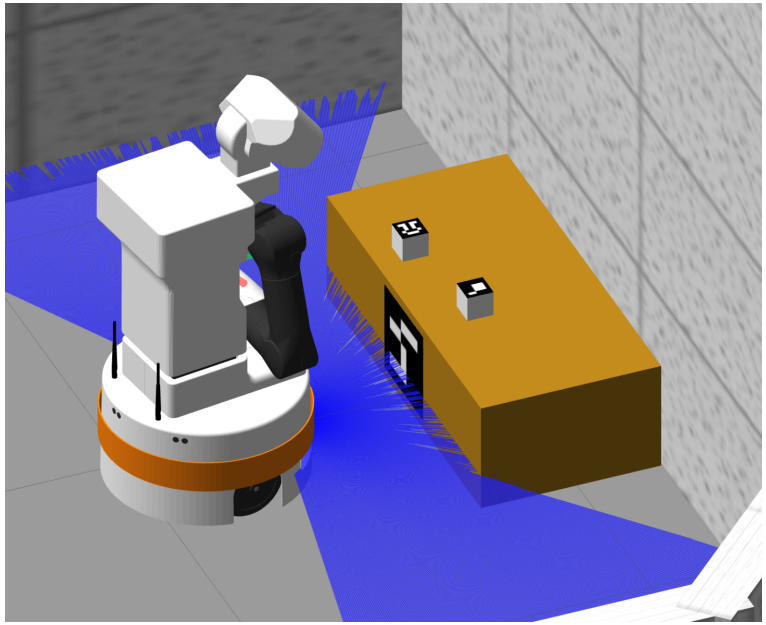


Figure 19: Head Scanning for Marker Detection

Once the robot head hass latched onto the marker, the robot stare at the marker for 1 second to ensure a stable pose estimation before executing the grasping sequence.

Task 3: Grasp & Release Selection Pose

Pose Calculation

The Grasp Pose is computed by extracting the marker's transform. The choice of the grasping point has been made to ensure a stable grip on the cube, considering collinear and antipodal grasp for the 7cm cube.

Pose Computation (KDL)

Given the marker pose ${}^{base}T_{marker}$, we compute the approach vector by applying a geometrical offset:

$$T_{grasp} = {}^{base}T_{marker} \cdot T_{rotY(90^\circ)} \cdot T_{offset(-0.4m)}$$

This aligns the gripper normal with the cube's face.

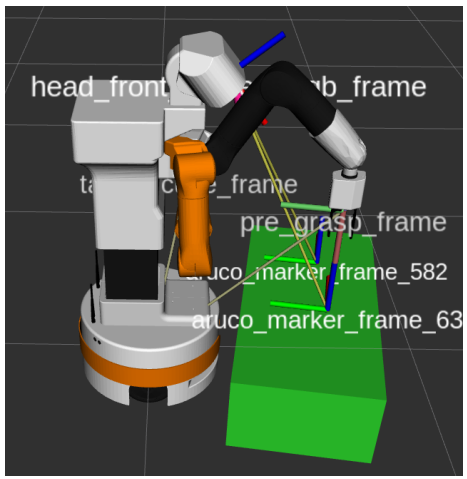


Figure 20: TF Frames visualization

Placement Logic: The release pose is derived from the grasp pose but lifted vertically to ensure the object rests on the table:

$$P_{release_z} = P_{grasp_z} + 0.15m$$

Task 3: Manipulation Safety & Robustness

The manipulation sequence follows a strict cycle:

$$\text{Pre_op} \rightarrow \text{op} \rightarrow \text{Post_op} \rightarrow \text{Safe Pose}$$

Motion Safety: Decoupled Retraction

The $0.4m$ approach offset requires the `arm_torso` group to extend significantly. To prevent the torso from lowering while the arm is still over the table (risk of collision), we implemented a **Decoupled Safe Procedure**:

1. **Arm First:** The arm retracts to a folded configuration while the torso remains fixed.
2. **Torso Second:** Only once the arm is safe, the torso lowers to the travel height.

Environmental Awareness: Upon arrival, **Collision Objects** representing the table (known dimensions) are spawned in MoveIt to forbid planning through the surface.

Task 3: Manipulation Safety & Robustness (Cont.)

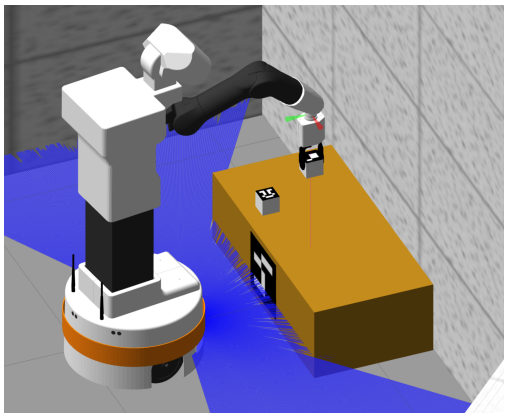


Figure 21: Cube Extraction

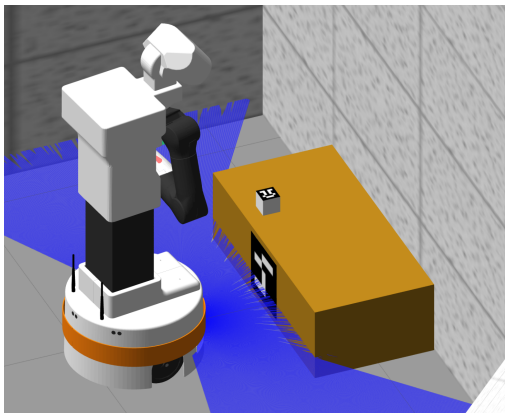


Figure 22: Arm Retraction

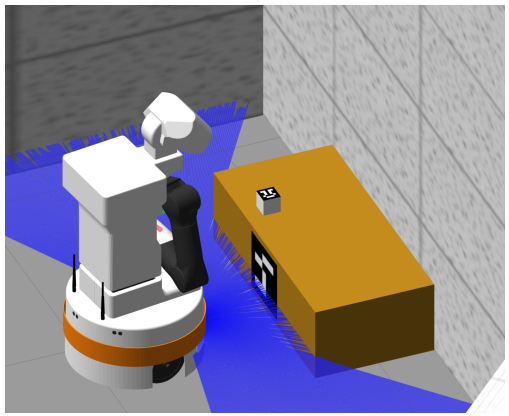


Figure 23: Torso Retraction

At the end of the manipulation operation the robot performs a *backup* maneuver to reach a safe distance from the table before rotating.

Task 3: Precision Placement Strategy

To ensure accurate object placement, we adopted a **Relative Pose Replication** strategy rather than absolute coordinate planning.

The Logic:

- The robot records the exact **base-to-table distance** (d_{ref}) during the successful Grasp phase.
- This distance is enforced as a constraint during the Placement phase to guarantee the **arm's kinematic reachability**.

Why?

Reusing the grasp pose relative to the base minimizes Inverse Kinematics (IK) errors and ensures the object lands exactly on the table surface.

Active Error Correction (Closed-Loop)

Since AMCL localization has a margin of error ($> 2cm$), the robot performs a final **Linear Approach/Backup** maneuver before placing:

$$e = d_{real} - d_{ref}$$

If $|e| > 1.0$cm: The base adjusts its position ($v_x = \pm 0.1$m/s) until convergence.

Task 3: Grasping Dynamics

Mobile manipulation in physics engines (Gazebo/ODE) suffers from unreliable **contact dynamics** and friction simulation, often leading to object slippage during base motion.

Attempt 1: Force Control (Failed)

We initially attempted a realistic grasp based on Force/Torque sensor feedback:

- **Tuning:** Adjusted friction coefficients (μ) and contact compliance (k_p, k_d).
- **Logic:** Stop gripper closure upon reaching a force threshold ($F_{th} = 10\text{N}$).

Outcome: Proven unreliable due to physics engine instability.

Solution: Kinematic Locking

To guarantee transport stability, we bypassed the physics engine using the **Link Attacher Plugin**.

- **Mechanism:** Creates a rigid kinematic joint (Fixed Joint) between the gripper and the object.
- **Result:** Zero slippage and robust transport regardless of base accelerations.

Task 3: Grasping Video Result

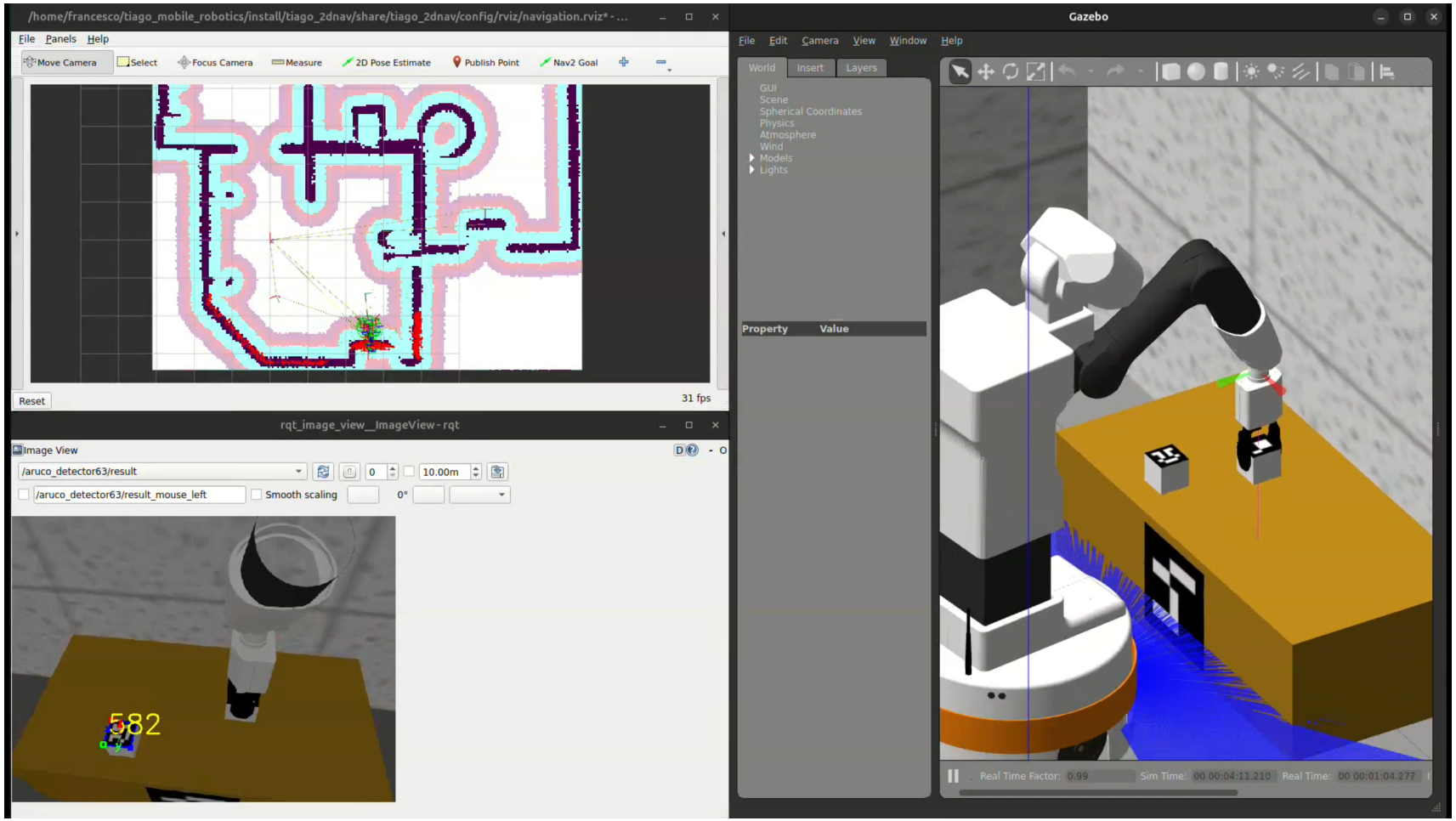


Figure 24: Manipulation Task Execution (Click to play in browser)